

Guard App — Background Location Tracking

Background location is the most critical technical feature of the guard app. Guards must continuously broadcast their GPS coordinates to the backend every 10 seconds — even when the app is minimized, the screen is locked, or the guard switches to another app.

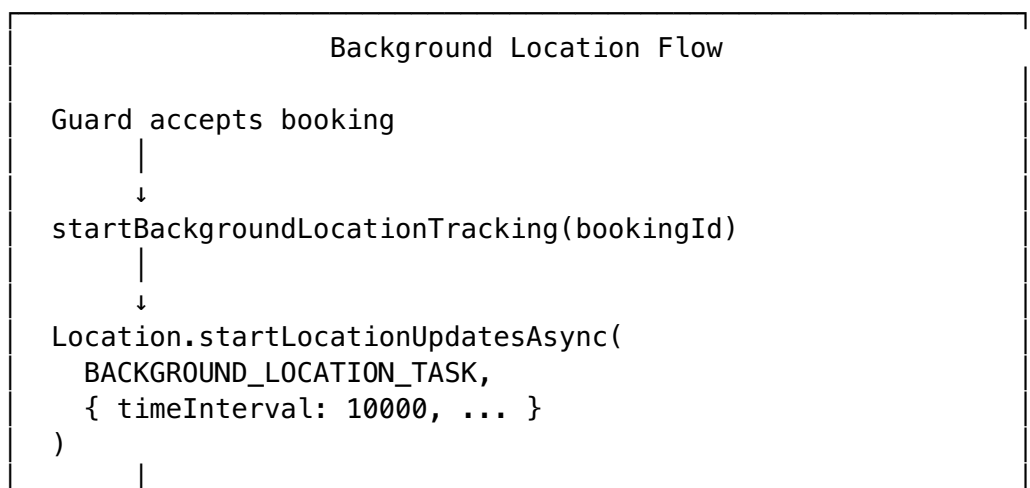
1. Why Background Location

When a guard has an active booking, the user and the operations team must be able to see the guard's real-time position on a map. This requires:

- Location updates every 10 seconds (not just when foregrounded)
- Updates even when the screen is locked
- Updates even when the guard uses another app
- Graceful handling when the device has poor GPS signal

This is implemented using **Expo's expo-location** with `startLocationUpdatesAsync`, which registers a native background task via **expo-task-manager**.

2. Architecture Overview



```

    ↓ (every 10 seconds, even backgrounded)
    TaskManager.defineTask handler fires
    ↓
    fetch POST /api/tracking/location/
    { lat, lon, heading, speed, booking_id }
    ↓
    Guard goes offline / booking completes
    ↓
    stopBackgroundLocationTracking()

```

3. Required Packages

`npx expo install expo-location expo-task-manager`

Verify in `package.json`:

```

{
  "expo-location": "~17.0.1",
  "expo-task-manager": "~12.0.1"
}

```

4. Task Definition

The task **must** be defined in the root of the app — before any React component renders. The best place is `app/_layout.tsx` or a dedicated `lib/backgroundLocation.ts` that is imported at the top level.

```

// lib/backgroundLocation.ts
import * as Location from 'expo-location';
import * as TaskManager from 'expo-task-manager';
import { BACKGROUND_LOCATION_TASK, API_URL } from './constants';

// — Task Definition

// IMPORTANT: Must be called at module load time, not inside a
// component.
TaskManager.defineTask(
  BACKGROUND_LOCATION_TASK,
  async ({ data, error }:
    TaskManager.TaskManagerTaskBody<{ locations:
      Location.LocationObject[] }>) => {
    if (error) {
      console.error('[BackgroundLocation] Task error:',
        error.message);
    }
  }
);

```

```

    return;
}

if (!data?.locations?.length) return;

const location = data.locations[0];
const { latitude, longitude, heading, speed, accuracy } =
  location.coords;

// Retrieve the stored token and active booking ID from
  AsyncStorage.
// We cannot use Zustand in background tasks – the JS context
  may be separate.
let token: string | null = null;
let bookingId: string | null = null;

try {
  const AsyncStorage = (await import('@react-native-async-
    storage/async-storage')).default;
  token = await AsyncStorage.getItem('auth_token');
  bookingId = await AsyncStorage.getItem('active_booking_id');
} catch {
  return; // Cannot proceed without auth
}

if (!token || !bookingId) return;

try {
  await fetch(`${API_URL}/tracking/location/`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify({
      booking_id: bookingId,
      lat: latitude,
      lon: longitude,
      heading: heading ?? null,
      speed: speed ?? null,
      accuracy: accuracy ?? null,
      timestamp: new Date(location.timestamp).toISOString(),
    }),
  });
} catch (fetchError) {
  // Silently fail – next tick will retry
  console.warn('[BackgroundLocation] Failed to send location
    update');
}
}
);

// — Start Tracking

```

```

export async function startBackgroundLocationTracking(bookingId:
  string): Promise<void> {
  // Store booking ID for background task access
  const AsyncStorage = (await import('@react-native-async-storage/
    async-storage')).default;
  await AsyncStorage.setItem('active_booking_id', bookingId);

  // Check if already running
  const isRunning = await
    Location.hasStartedLocationUpdatesAsync(BACKGROUND_LOCATION_TASK);
  if (isRunning) return;

  await Location.startLocationUpdatesAsync(BACKGROUND_LOCATION_TASK,
    {
      accuracy: Location.Accuracy.Balanced, // Battery-
        efficient
      timeInterval: 10_000, // Minimum 10s
        between updates
      distanceInterval: 20, // OR 20 meters
        of movement (whichever fires first)
      deferredUpdatesInterval: 10_000,
      deferredUpdatesDistance: 20,
      showsBackgroundLocationIndicator: true, // iOS: shows
        blue bar
      foregroundService: { // Android:
        required for background operation
        notificationTitle: 'b-secure Guard is active',
        notificationBody: 'Sharing your location during active
          booking',
        notificationColor: '#1B4332',
      },
      pausesUpdatesAutomatically: false, // Never pause
      activityType: Location.ActivityType.AutomotiveNavigation,
    });

  console.log('[BackgroundLocation] Started for booking:',
    bookingId);
}

// — Stop Tracking

export async function stopBackgroundLocationTracking():
  Promise<void> {
  const AsyncStorage = (await import('@react-native-async-storage/
    async-storage')).default;
  await AsyncStorage.removeItem('active_booking_id');

  const isRunning = await
    Location.hasStartedLocationUpdatesAsync(BACKGROUND_LOCATION_TASK);
  if (!isRunning) return;

  await Location.stopLocationUpdatesAsync(BACKGROUND_LOCATION_TASK);
  console.log('[BackgroundLocation] Stopped');
}

```

```
// — Check if Running
```

```
export async function isBackgroundLocationRunning():  
  Promise<boolean> {  
    return  
      Location.hasStartedLocationUpdatesAsync(BACKGROUND_LOCATION_TASK);  
  }  
}
```

5. Permissions

Always request foreground permission first, then background. Both are required. Request them during onboarding or before the guard first goes online.

```
// lib/locationPermissions.ts  
import * as Location from 'expo-location';  
import { Alert, Platform, Linking } from 'react-native';  
  
export async function requestLocationPermissions():  
  Promise<boolean> {  
    // Step 1: Foreground permission  
    const { status: foregroundStatus } =  
      await Location.requestForegroundPermissionsAsync();  
  
    if (foregroundStatus !== 'granted') {  
      Alert.alert(  
        'Location Permission Required',  
        'b-secure Guard needs location permission to function.  
        Please enable it in Settings.',  
        [  
          { text: 'Cancel', style: 'cancel' },  
          { text: 'Open Settings', onPress: () =>  
            Linking.openSettings() },  
        ],  
      );  
      return false;  
    }  
  
    // Step 2: Background permission (Android 10+ / iOS 13+)  
    if (Platform.OS === 'android' && Platform.Version < 29) {  
      // Android < 10: background access is automatically granted  
      // with foreground  
      return true;  
    }  
  
    const { status: backgroundStatus } =  
      await Location.requestBackgroundPermissionsAsync();  
  
    if (backgroundStatus !== 'granted') {  
      Alert.alert(  
        'Background Location Required',
```

```

Platform.OS === 'ios'
  ? 'Please set location access to "Always" in Settings to
    enable background tracking during bookings.'
  : 'Please enable "Allow all the time" location access in
    Settings to track your location during bookings.',
[
  { text: 'Later', style: 'cancel' },
  { text: 'Open Settings', onPress: () =>
    Linking.openSettings() },
]
);
return false;
}

return true;
}

export async function checkLocationPermissions(): Promise<{
  foreground: boolean;
  background: boolean;
}> {
  const { status: foreground } = await
    Location.getForegroundPermissionsAsync();
  const { status: background } = await
    Location.getBackgroundPermissionsAsync();
  return {
    foreground: foreground === 'granted',
    background: background === 'granted',
  };
}

```

6. app.json Native Configuration

```

{
  "expo": {
    "plugins": [
      [
        "expo-location",
        {
          "locationAlwaysAndWhenInUsePermission": "b-secure Guard
            needs your location at all times to share your position
            with users during active bookings.",
          "locationAlwaysPermission": "b-secure Guard needs your
            location in the background during active bookings.",
          "locationWhenInUsePermission": "b-secure Guard needs
            your location to show you on the map.",
          "isIosBackgroundLocationEnabled": true,
          "isAndroidBackgroundLocationEnabled": true
        }
      ],
      "expo-task-manager"
    ]
  }
}

```

```

],
"ios": {
  "infoPlist": {
    "NSLocationAlwaysAndWhenInUseUsageDescription": "b-secure Guard needs your location at all times to share your position with users during active bookings.",
    "NSLocationAlwaysUsageDescription": "b-secure Guard requires always-on location access during active security bookings.",
    "NSLocationWhenInUseUsageDescription": "b-secure Guard needs your location to show your position on the map.",
    "UIBackgroundModes": [
      "location",
      "fetch",
      "remote-notification"
    ]
  }
},
"android": {
  "permissions": [
    "ACCESS_FINE_LOCATION",
    "ACCESS_COARSE_LOCATION",
    "ACCESS_BACKGROUND_LOCATION",
    "FOREGROUND_SERVICE",
    "FOREGROUND_SERVICE_LOCATION"
  ],
  "foregroundService": {
    "notificationTitle": "b-secure Guard Active",
    "notificationBody": "Tracking location during booking",
    "notificationColor": "#1B4332"
  }
}
}
}
}

```

7. Integration with Guard Online Status

The location tracking lifecycle is tied to booking lifecycle — **not** the online/offline toggle. Start tracking when a booking becomes active; stop when it completes.

```

// hooks/useGuardStatus.ts (relevant section)
import { startBackgroundLocationTracking,
  stopBackgroundLocationTracking } from '@lib/
  backgroundLocation';
import { requestLocationPermissions } from '@lib/
  locationPermissions';

// When guard goes online – only request permissions, don't start
  tracking yet
async function handleGoOnline() {

```

```

const hasPermission = await requestLocationPermissions();
if (!hasPermission) return;

await guardProfileService.setOnlineStatus(true);
setOnline(true);
}

// When booking is accepted – START background tracking
async function handleBookingAccepted(bookingId: string) {
  await startBackgroundLocationTracking(bookingId);
}

// When booking completes – STOP background tracking
async function handleBookingCompleted() {
  await stopBackgroundLocationTracking();
}

```

8. Battery Optimization

Setting	Value	Reason
accuracy	Location.Accuracy.Balanced	Uses cell towers + WiFi, not pure GPS. Saves ~30-40% battery vs High.
timeInterval	10_000 ms	10 second minimum between updates
distanceInterval	20 meters	Also fires if guard moves 20m, even before 10s
activityType	AutomotiveNavigation	Hints to OS that movement is expected; prevents aggressive suspension

Setting	Value	Reason
<code>pausesUpdatesAutomatically</code>	<code>false</code>	Prevents iOS from pausing updates when guard is stationary

Only start tracking when a booking is active. Do not track when the guard is online but idle — this wastes battery unnecessarily.

9. Sending Location via WebSocket (Alternative)

When the guard app is in the foreground and the WebSocket is connected, it is more efficient to send location updates via WebSocket to reduce HTTP overhead:

```
// In the background task – use HTTP (WebSocket not available in
  background)
// In the foreground – the WebSocket service can send location
  updates

// services/websocket/guardWebSocketService.ts
sendLocationUpdate(lat: number, lon: number, heading: number |
  null, speed: number | null, bookingId: string) {
  this.send({
    type: 'location.update',
    payload: { lat, lon, heading, speed, booking_id: bookingId },
  });
}
```

The background task always uses fetch (HTTP) because WebSocket connections are not maintained in the background on iOS or Android.

10. Complete Background Location Module

The complete, production-ready module combining all of the above:

```
// lib/backgroundLocation.ts
import * as Location from 'expo-location';
import * as TaskManager from 'expo-task-manager';
import AsyncStorage from '@react-native-async-storage/async-storage';
```

```
import { BACKGROUND_LOCATION_TASK, API_URL } from './constants';
```

```
// — Storage Keys
```

```
const AUTH_TOKEN_KEY = 'auth_token';
```

```
const ACTIVE_BOOKING_KEY = 'active_booking_id';
```

```
// — Task Registration (must be top-level)
```

```
TaskManager.defineTask(
  BACKGROUND_LOCATION_TASK,
  async ({
    data,
    error,
  }: TaskManager.TaskManagerTaskBody<{
    locations: Location.LocationObject[];
  }>) => {
    if (error) {
      console.error('[BGLocation]', error.message);
      return;
    }

    if (!data?.locations?.length) return;

    const location = data.locations[data.locations.length -
      1]; // Latest location
    const { latitude, longitude, heading, speed, accuracy } =
      location.coords;

    let token: string | null = null;
    let bookingId: string | null = null;

    try {
      [token, bookingId] = await Promise.all([
        AsyncStorage.getItem(AUTH_TOKEN_KEY),
        AsyncStorage.getItem(ACTIVE_BOOKING_KEY),
      ]);
    } catch {
      return;
    }

    if (!token || !bookingId) return;

    const payload = {
      booking_id: bookingId,
      lat: latitude,
      lon: longitude,
      heading: heading !== undefined ? heading : null,
      speed: speed !== undefined ? speed : null,
      accuracy: accuracy !== undefined ? accuracy : null,
      timestamp: new Date(location.timestamp).toISOString(),
    };
  }
);
```

```

    try {
      const response = await fetch(`${API_URL}/tracking/location/`, {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          Authorization: `Bearer ${token}`,
        },
        body: JSON.stringify(payload),
        signal: AbortSignal.timeout(8000), // 8s timeout
      });

      if (!response.ok) {
        console.warn('[BGLocation] HTTP error:', response.status);
      }
    } catch (err) {
      console.warn('[BGLocation] Network error – will retry next tick');
    }
  }
);

```

// — Public API

```

export async function startBackgroundLocationTracking(
  bookingId: string
): Promise<{ success: boolean; error?: string }> {
  try {
    // Persist booking ID for background task
    await AsyncStorage.setItem(ACTIVE_BOOKING_KEY, bookingId);

    // Guard against double-start
    const alreadyRunning = await
      Location.hasStartedLocationUpdatesAsync(
        BACKGROUND_LOCATION_TASK
      );
    if (alreadyRunning) {
      return { success: true };
    }

    await
      Location.startLocationUpdatesAsync(BACKGROUND_LOCATION_TASK,
        {
          accuracy: Location.Accuracy.Balanced,
          TimeInterval: 10_000,
          distanceInterval: 20,
          deferredUpdatesInterval: 10_000,
          deferredUpdatesDistance: 20,
          showsBackgroundLocationIndicator: true,
          foregroundService: {
            notificationTitle: 'b-secure Guard – Active Booking',
            notificationBody: 'Sharing your location with the user',

```

```

        notificationColor: '#1B4332',
    },
    pausesUpdatesAutomatically: false,
    activityType: Location.ActivityType.AutomotiveNavigation,
});

    return { success: true };
} catch (error: any) {
    console.error('[BGLocation] Failed to start:', error);
    return { success: false, error: error.message };
}
}

export async function stopBackgroundLocationTracking():
    Promise<void> {
    try {
        await AsyncStorage.removeItem(ACTIVE_BOOKING_KEY);

        const isRunning = await
            Location.hasStartedLocationUpdatesAsync(
                BACKGROUND_LOCATION_TASK
            );
        if (isRunning) {
            await
                Location.stopLocationUpdatesAsync(BACKGROUND_LOCATION_TASK);
        }
    } catch (error) {
        console.error('[BGLocation] Failed to stop:', error);
    }
}

export async function isLocationTrackingActive():
    Promise<boolean> {
    try {
        return await
            Location.hasStartedLocationUpdatesAsync(BACKGROUND_LOCATION_TASK);
    } catch {
        return false;
    }
}

export async function getCurrentLocation():
    Promise<Location.LocationObject | null> {
    try {
        const { status } = await
            Location.getForegroundPermissionsAsync();
        if (status !== 'granted') return null;

        return await Location.getCurrentPositionAsync({
            accuracy: Location.Accuracy.Balanced,
        });
    } catch {
        return null;
    }
}

```

```
}  
}
```

11. Testing Background Location

iOS Simulator

The iOS Simulator supports simulated location but not true background location. Use Xcode's built-in location simulation: - **Xcode** → **Debug** → **Simulate Location** for basic testing

Physical Device Testing Checklist

☐

Grant “Allow Always” location permission in iOS Settings → Privacy → Location Services → b-secure Guard

☐

Enable Android's “Allow all the time” in App Settings → Permissions → Location

☐

Disable Battery Optimization for the app on Android (Settings → Battery → App Optimization)

☐

Lock the screen and confirm location updates continue arriving at the backend

☐

Force-minimize the app (swipe up / home button) and confirm updates continue

☐

Switch to another app and confirm updates continue

☐

Kill the app entirely — updates should STOP (this is expected behavior; task-manager does not survive app kill on most devices)

Debugging

```
// Check task registration  
const registeredTasks = await  
    TaskManager.getRegisteredTasksAsync();  
console.log('Registered tasks:', registeredTasks);  
  
// Check if our task is registered  
const isRegistered = await  
    TaskManager.isTaskRegisteredAsync(BACKGROUND_LOCATION_TASK);
```

```
console.log('BGLocation task registered:', isRegistered);

// Check if task is running
const isRunning = await
    Location.hasStartedLocationUpdatesAsync(BACKGROUND_LOCATION_TASK);
console.log('BGLocation running:', isRunning);
```